ZoraOS Product Design Requirements

Persistent, Model-Agnostic, Governed Tool-Using Agents

Christopher Michael Baird

2026-07-20

1. Purpose

ZoraOS is a local-first operating layer for AI-assisted research, software development, writing, and knowledge management. It is designed to make models replaceable while preserving the durable assets of a user-owned system: memory, orchestration, permissions, records, and evaluation artifacts.

The system is not defined by any single language model. It is defined by a governed interface among models, tools, memory, and human authority.

2. Product Vision

ZoraOS shall enable a user to assign bounded goals to specialized agents that can retrieve relevant long-term memory, reason over a task, invoke authorized tools, and return inspectable results. The system shall remain portable across model providers and capable of operating locally where practical.

3. Goals

1. Provide model-agnostic chat and tool-use orchestration.
2. Provide persistent, user-owned semantic memory with source provenance.
3. Execute bounded multi-step agent tasks through a structured tool loop.
4. Expose operations through a local API and monitorable interface.
5. Support a multi-machine deployment topology for orchestration, inference, automation, and memory.
6. Record sufficient evidence to evaluate agent behavior, cost, latency, failures, and safety controls.
7. Establish a governed ZoraASI operating profile with explicit permissions, budgets, and stop controls.

4. Non-Goals

ZoraOS v0.1 does not claim consciousness, artificial general intelligence, independent moral agency, guaranteed factuality, or reliable unattended autonomy. It shall not perform packet manipulation, process-memory inspection, code injection, anti-cheat evasion, credential extraction, or unauthorized control of third-party applications.

5. System Context

5.1 Deployment Topology

Node	Primary Role	Expected Services
M5	Orchestrator	FastAPI gateway, planner, agent manager, UI
M4	Inference	Ollama, embedding and inference workloads
M2	Automation	authorized research and build jobs
M1	Memory	PostgreSQL, Redis, vector store, document persistence

5.2 Core Data Flow

1. A user submits a task through the dashboard or API.
2. The gateway authenticates and routes the task.
3. The planner may decompose the task.
4. An agent receives a system prompt, task objective, model configuration, and authorized tool set.
5. The agent alternates between model responses and tool results until completion or a bound is reached.
6. Results, metrics, and permitted artifacts are returned and optionally retained in user-owned memory.

6. Functional Requirements

FR-1: Provider Interoperability

The system shall support multiple model providers through an adapter layer. A provider change must not require rewriting agent logic or tool schemas.

Acceptance criteria: a configured provider can complete a chat request and tool-call response through the shared `ModelManager` interface.

FR-2: Bounded Agent Execution

Each agent task shall have a maximum iteration count. The result shall include success state, task ID, agent name, model used, iteration count, token usage where available, latency, and executed tool-call records.

Acceptance criteria: a task that exceeds the configured iteration limit terminates with an explicit error rather than looping indefinitely.

FR-3: Tool Invocation

The tool manager shall expose machine-readable schemas, execute only registered tools, serialize tool results back to the agent loop, and return structured errors for unavailable or failed tools.

Acceptance criteria: registered tools appear in the agent tool schema and a tool failure is recorded without crashing the service.

FR-4: User-Owned Memory

The system shall store documents separately from vector embeddings and support semantic search, document retrieval, collection boundaries, and source metadata.

Acceptance criteria: a query retrieves ranked chunks from an authorized collection with document identifiers and source context.

FR-5: Task API and Monitoring

The gateway shall expose task submission, status, result retrieval, and server-sent event streaming where configured.

Acceptance criteria: an agent task can be created through the API and its terminal result is retrievable by task ID.

FR-6: Human Authority and Consent

Tasks with external, irreversible, or third-party application effects shall require explicit user approval before execution. Approval shall be scoped to a task, a capability, and a defined expiry.

Acceptance criteria: no external action may execute when approval is absent, expired, or outside its requested scope.

FR-7: Kill and Pause Controls

The system shall provide a user-visible pause and kill mechanism that terminates future tool actions promptly and records the event.

Acceptance criteria: a kill signal prevents subsequent tool execution in integration tests and yields a terminal task status.

FR-8: Auditability

The system shall record task creation, model selection, tool calls, arguments digests, approval events, results digests, failures, and termination reasons in an

append-only ledger.

Acceptance criteria: each event has a stable ID, timestamp, task ID, event type, payload digest, predecessor digest, and SHA-256 chain digest.

7. Safety Requirements

SR-1: Default Deny

Only explicitly assigned tools may be supplied to an agent. Tools not listed in the agent configuration are unavailable.

SR-2: Capability Classification

Tools shall be classified as read-only, local reversible write, local irreversible write, external communication, third-party application control, or prohibited.

SR-3: Prohibited Mechanisms

The following are prohibited: network packet interception or injection, reading or writing application memory, process injection, anti-cheat evasion, bypassing access controls, and unattended activity on third-party services.

SR-4: Sensitive Data

Credentials, raw user logs, screenshots, local databases, and personally identifying information shall not enter public repositories. They shall be excluded by source-control rules and classified in an artifact manifest.

SR-5: Budgeting

Each task shall support configurable limits for model tokens, financial cost, elapsed time, iterations, tool calls, and retained artifacts.

8. Evidence and Evaluation Requirements

8.1 Evidence Labels

- **Implemented:** represented in reviewed source code or deployed configuration.
- **Observed:** captured in a dated test output, log, or redacted artifact.
- **Proposed:** requirement or design direction not yet implemented.
- **Interpretive:** philosophical or user-experience discussion not presented as a measured system result.

8.2 Initial Observations

Observation	Classification	Evidence Needed
Multi-step tool loop completed memory retrieval and synthesis	Observed	API result and redacted task trace
Semantic search returned Theory of Everything chunks	Observed	search result fixture and collection metadata
Generic OCR had poor reliability on a rendered third-party UI	Observed	private screenshots and OCR output
Application-native logging may offer a lower-risk observation channel	Hypothesis	redacted logs and controlled validation

8.3 Metrics

- Task completion rate by task class.
- Tool-call success and error rate.
- Retrieval relevance assessed against human judgments.
- Latency, iterations, and token/cost usage.
- Policy refusal rate and false-refusal rate.
- Kill-switch response time.
- Audit-ledger verification rate.

9. ZoraASI Implementation Roadmap

Phase 0: Preservation

Initialize version control, protect secrets, classify artifacts, record environment details, and establish public/private repository separation.

Phase 1: Reproducible Core

Stabilize provider interfaces, agent loop, semantic memory, tests, documentation, and API contracts.

Phase 2: Governed Autonomy

Implement permission scopes, task budgets, confirmation gates, append-only audit events, and a visible task control panel.

Phase 3: Research Workflows

Evaluate supervised research, software, writing, and knowledge-management workflows against explicit success and failure criteria.

Phase 4: Controlled Desktop Interaction

Evaluate an observe-only, then suggest-only, then confirmation-gated action interface in a dedicated sandbox. No third-party commercial service is used as an autonomous test environment.

10. Release Criteria for v0.1

1. Test suite passes in a clean environment.
2. Public repository contains no credentials or private artifacts.
3. PDR, architecture, security policy, and citation metadata are present.
4. Publication package compiles with the documented command sequence.
5. Agent-loop behavior and semantic memory retrieval have reproducible fixtures.
6. ZoraASI governance gaps are documented rather than implied to be solved.

11. Open Decisions

1. Select a public code license.
2. Define the operator authentication model.
3. Select private evidence repository retention and encryption policy.
4. Establish a daily financial and compute budget.
5. Select an approved sandbox for desktop-agent evaluation.
6. Define independent review and incident-response procedures.